

Modurile de adresare ale procesorului 80386

Scopul lucrării

În cadrul acestei lucrări se prezintă modurile de adresare permise de procesorul 80386. Partea aplicativă a lucrării are drept scop exersarea acestor mecanisme de adresare și identificarea celor mai adecvate soluții de adresare a diferitelor structuri de date.

Prezentarea modurilor de adresare

Prin "mod de adresare" se înțelege un anumit mecanism folosit pentru determinarea operanzilor unei instrucțiuni. Așa cum se va vedea în continuare, un anumit operand care participă la execuția unei instrucțiuni se poate regăsi în diferite feluri: poate fi o constantă, un registru intern al procesorului, o locație de memorie sau un port al unei interfețe. Un anumit mod de adresare indică mecanismul de căutare a operandului și eventual modul de calcul a adresei, dacă operandul se află în memorie.

Diferitele moduri de adresare oferă suportul necesar pentru regăsirea elementelor unei structuri mai complexe de date. De exemplu adresarea indexată permite adresarea elementelor unui vector, sau adresarea bazată permite extragerea unor date dintr-o structură de tip înregistrare.

Mecanismul de adresare folosit influențează viteza de execuție a instrucțiunilor și determină lungimea acestora. Astfel mecanismele complexe de adresare necesită un timp de execuție mai mare însă oferă o mai mare flexibilitate în regăsirea datelor.

Adresarea imediată

Adresarea imediată este cea mai simplă formă de adresare. Operandul este o constantă, care se păstrează în codul instrucțiunii. Astfel, odată cu citirea instrucțiunii are loc și citirea operandului. Constanta se poate exprima în zecimal (este forma implicită), în hexazecimal (cu terminația "h"), în binar (terminația "b") sau sub formă de coduri ASCII. Constanta este întotdeauna al doilea operand al unei instrucțiuni.

Exemple:

```
mov    eax, 1234h
add    cx, 30
and    bh, 01111b
```

Acest mod de adresare este relativ rapid deoarece nu necesită transfer suplimentar pentru aducerea operandului. dar flexibilitatea este limitată, în sensul că o instrucțiune operează cu o singură valoare.

Adresarea de tip registru

La această adresare operandul se află într-un registru al procesorului. Acest mod este de fapt o formă mai eficientă de adresare directă. Eficiența se datorează mai multor factori:

- registrele sunt în interiorul procesorului ceea ce elimină necesitatea unui transfer suplimentar cu memoria
- adresa unui registru se exprimă pe un număr redus de biți (3-4 biți), ceea ce contribuie la reducerea dimensiunii instrucțiunilor și implicit la o execuție mai rapidă a acestora

- transferurile între registre se face la o viteză mult mai mare deoarece se utilizează magistralele interne ale procesorului

Exemple:

```
mov  ax, bx
cmp  ah,bl
mov  ax, ds
mov  es, ax
```

Nu sunt permise transferuri directe între registrele segment ale procesorului, sau între un registru segment și o constantă. De aceea încărcarea unui registru segment se face indirect prin intermedierea unui registru general.

Exemplu:

```
mov  ax, data_segment
mov  ds, ax
```

Dezavantajul acestui mod constă în faptul că numărul de registre interne este limitat și din această cauză nu toate variabilele unui program pot fi păstrate în registre.

Adresarea directă

Adresarea directă presupune prezența adresei operandului în codul instrucțiunii. Operandul este o locație de memorie sau cu alte cuvinte o variabilă. Adresa operandului se poate exprima printr-o valoare sau printr-un nume simbolic dat variabilei. Pentru a evita confuzia cu adresarea imediată, valoarea adresei se plasează între paranteze drepte. De altfel ori de câte ori se folosesc paranteze drepte, conținutul lor trebuie interpretat ca și o adresă și nu ca o constantă.

Exemple:

```
mov  ax, DS:[403000h]    ; 100h este adresa de offset a operandului
add  ES:[405001h], ch
cmp  dx, var16           ; var16 trebuie declarat în prealabil ca și o variabilă pe 16 biți
```

Adresarea directă permite accesarea unei singure locații de memorie; o altă locație necesită o altă instrucțiune.

Moduri de adresare indirectă

În cazul modurilor de adresare indirectă instrucțiunea nu conține operandul sau adresa acestuia ci o indicație asupra locului unde se află adresa operandului. De cele mai multe ori adresa se păstrează într-un registru sau se calculează ca o sumă de registre și eventual o constantă. Deși inițial, la procesoarele 8086 exista un număr restrâns de registre care puteau fi utilizate pentru a păstra adrese, și anume SI, DI pentru adresarea indexată și BX, BP pentru adresare bazată, începând cu procesoarele 80386 pot fi folosite pentru adresare toate registrele generale pe 32 de biți, și anume EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP. În cazul registrelor ESP și EBP registrul segment folosit în mod implicit pentru adresare este SS, pentru celelalte registre se va folosi implicit DS.

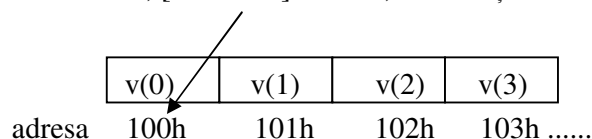
Modurile de adresare indirectă la procesoarele 80386 sunt: adresarea indexată, adresarea bazat-indexată și adresarea scalat-indexată.

Adresarea indexată

Adresarea indexată permite utilizarea unui registru pe 32 de biți (indexul) și a unei constante (reprezentând deplasamentul) pentru calculul adresei. Deplasamentul indică adresa de început a vectorului, iar registrul index poziția relativă a elementului față de adresa de început. Acest mod de adresare este utilizat pentru regăsirea elementelor unei structuri de tip vector, prin simpla incrementare sau decrementare a unui registru denumit registru index. O singură instrucțiune poate prelucra toate elementele unui vector.

Exemple:

```
mov    ax, [esi]
and     var[esi], 30h      ; var se traduce printr-o constantă care se adună la ESI
mov     [edi], cl
add     ah, [esi+100h]     ; ESI conține indexul elementului căutat
```



```
mov    ax, [ebp]
add     [ebx+100h], 50
sub     dx, var[ecx]
```

Trecerea la elementul următor din vector se face prin incrementarea sau decrementarea explicită a registrului index. Dacă elementele vectorului sunt octeți atunci factorul de incrementare este 1 iar dacă sunt cuvinte atunci factorul este 2, etc..

Acest mod de adresare este mai puțin eficient din punct de vedere al vitezei de execuție deoarece implică un calcul matematic și o adresare suplimentară a memoriei. În schimb este o metodă flexibilă de adresare a structurilor de tip vector sau tablou. În cazul unor prelucrări în buclă aceeași instrucțiune adresează succesiv toate elementele unui vector.

Adresarea bazat-indexată

Adresarea bazat-indexată permite utilizarea a două registre pe 32 de biți (baza și indexul) și a unei constante (reprezentând deplasamentul) pentru calculul adresei. Primul registru utilizat este registrul baza, al doilea este registrul index. Este permis să se folosească același registru atât ca baza cât și ca index. În funcție de registrul baza utilizat se va folosi implicit registrul segment SS (dacă registrul baza este EBP sau ESP) sau DS (în celelalte cazuri).

Acest mod se utilizează în cazul unor structuri de date complexe de tip vectori de înregistrări, înregistrări de vectori, tabele bidimensionale, etc. Este o formă mai flexibilă de adresare dar în același timp ineficientă. Adresarea operandului implică două adunări și un transfer suplimentar cu memoria.

Exemple:

```
mov     ax, [ebx][esi]      ;se folosește implicit DS
sub     var[ebp+esi],ch     ;se folosește implicit SS
add     ax, [eax+ecx+100h]
```

Adresarea indexat-scalată

Adresarea indexat-scalată permite utilizarea a două registre pe 32 de biți (baza și indexul) și a unei constante (reprezentând deplasamentul) pentru calculul adresei. În acest mod de adresare este permisă înmulțirea registrului index cu o valoare egală cu 1, 2, 4 sau 8.

Exemple:

```
mov    ax, [esi*2]
sub    1000h[ebx][edi*8],ch
add    ax, 1000h[esi*4],
```

Adresarea pe șiruri

Adresarea pe șiruri este o formă specială de adresare indexată. La această adresare registrele index folosite nu se precizează, ele fiind definite în mod implicit: ESI – pentru șirul sursă și EDI pentru șirul destinație. În plus registrul ECX este folosit pe post de contor. La execuția instrucțiunii are loc modificarea automată (prin incrementare sau decrementare) a registrelor index astfel încât să se treacă automat la elementele următoare din șir. Prin utilizarea prefixului de repetare (REP, REPZ,..) se obține transferul sau prelucrarea unui bloc de date printr-o singură instrucțiune. Este o metodă elegantă de lucru cu structuri de date de tip șir sau vector.

Exemplu:

```
lea    esi, sir1
lea    edi, sir2
mov    ecx, lung_sir
rep    movsb
```

Adresarea de tip stivă

Adresarea de tip stivă utilizează în mod implicit registrul ESP pentru adresarea unuia dintre operanzi. Acest mod de adresare este folosit numai la 2 instrucțiuni, cele care operează cu vârful stivei: PUSH și POP. În urma transferului registrul ESP se modifică în așa fel încât la următoarea operație să se adreseze din nou vârful stivei. La arhitectura ISA x86 stiva crește către adrese mai mici (ESP se decrementează la salvarea pe stivă) și descrește către adrese mai mari (ESP se incrementează la descărcarea unui element de pe stivă).

Exemple:

```
push   ax
push   var32
pop    ecx
```

Mersul lucrării

1. Discuții legate de problemele întâmpinate în înțelegerea documentației scrise.
2. Problema rezolvată în s4ex1.asm

Sa se scrie un program care copiază un șir de valori din locații consecutive de memorie în alta locație aflată în alt segment de date decât primul șir, în ordine inversă.

- a. Se va compila fișierul sursa
- b. Se va executa în Ollydbg
- c. Se vor urmări schimbările care au loc la nivelul regiștrilor, memoriei și a flagurilor

3. Sa se scrie un program care calculează media unui sir de numere întregi din memorie.

Numerele sunt de tip octet. Media va fi memorata ca valoare întreaga într-o variabila de tip octet.

4. Problema rezolvata in s4ex2.asm

Sa se scrie un program pentru adunarea a doua matrice bidimensionale.

- a. Se va compila fişierul sursa
- b. Se va executa in Ollydbg
- c. Se vor urmări schimbările care au loc la nivelul regiştrilor, memoriei si a flagurilor

5. Sa se scrie un program care însumează salariile unor angajaţi. Datele referitoare la angajaţi se păstrează într-un tablou de înregistrări.

Exemplu tablou înregistrări:

Angajat struct

nume db 20 dup(0)

salariu dw 0

Angajat ends

Angajati Angajat {"georgescu",100},{ "pop",100}, {"popescu",300},{ "ionescu",450}